

Create — Inserting Data

1. Explanation of Elements

To add data to our database, we use insert operations. Documents are written in BSON (which looks exactly like JSON).

- **insertOne()**: Adds a single document to the collection.
- **insertMany()**: Takes an array [...] and adds multiple documents at once.

2. Setup Data (Raw JSON to use in the queries)

```
JavaScript
// Single User Data
{ name: "John Doe", email: "john@test.com", age: 28 }

// Multiple Users Data
[
  { name: "Jane Smith", email: "jane@test.com", age: 32 },
  { name: "Mike Johnson", email: "mike@test.com", age: 24 }
]
```

3. Instructor Scenario: Registering a New User

Scenario: A user fills out a registration form on your website. You need to save their profile into the database.

The Query:

```
JavaScript
db.users.insertOne({
  name: "John Doe",
  email: "john@test.com",
  age: 28
})
```

4. Student Exercise

- ❏ **Scenario:** The HR department hands you a spreadsheet with three new employees that need to be added to the system all at once. Write the query to add these three employees to the `employees` collection in a single command.

Read — Finding Data (find vs findOne)

1. Explanation of Elements

To retrieve data, we use read operations. We pass a "filter" object to tell MongoDB what to look for.

- **find():** Returns a list of all documents that match the filter.
- **findOne():** Returns only the very first document that matches the filter as a single object.

2. Setup Data (Run this first!)

```
JavaScript
db.books.insertMany([
  { title: "The Hobbit", author: "J.R.R. Tolkien", genre: "Fantasy" },
  { title: "1984", author: "George Orwell", genre: "Sci-Fi" },
  { title: "Dune", author: "Frank Herbert", genre: "Sci-Fi" }
])
```

3. Instructor Scenario: The Book Search

Scenario: A user searches your library for the exact book "The Hobbit". Because there is only one specific book with this title, we want the actual book object back, not a list.

The Query:

```
JavaScript
db.books.findOne({ title: "The Hobbit" })
```

4. Student Exercise

📌 **Scenario:** A user clicks on the "Sci-Fi" category on your bookstore homepage. Write the query to return a list of ALL books that have "Sci-Fi" as their genre.

Update — Modifying Data

1. Explanation of Elements

To change existing data, we must provide two things: a filter (who to update) and an update operator (what to change). We use the `$set` operator to change specific fields.

- **updateOne()**: Modifies the first document that matches the filter. Returns an acknowledgment status.
- **findOneAndUpdate()**: Modifies the document and returns the actual document back to you.

2. Setup Data (Run this first!)

```
JavaScript
db.orders.insertMany([
  { orderId: "101", customer: "Alice", status: "processing", total: 50 },
  { orderId: "102", customer: "Bob", status: "processing", total: 120 }
])
```

3. Instructor Scenario: Order Shipped!

Scenario: The warehouse just shipped Alice's order (orderId "101"). We need to update her specific order status from "processing" to "shipped".

The Query:

```
JavaScript
db.orders.updateOne(
  { orderId: "101" }, // 1. The Filter: Find Alice's order
  { $set: { status: "shipped" } } // 2. The Update: Change the status
)
```

4. Student Exercise

- ❏ **Scenario:** You are building an admin panel and need to update Bob's order (orderId "102"). The total was calculated wrong; it should be 150. Write the query to update his total, AND use the method that will return his updated document back to you immediately.

Delete — Removing Data

1. Explanation of Elements

Sometimes data needs to be permanently removed. Like finding and updating, this requires a filter.

- **deleteOne()**: Deletes the first document that matches the filter.
- **deleteMany()**: Deletes ALL documents that match the filter.

2. Setup Data (Run this first!)

```
JavaScript
db.accounts.insertMany([
  { username: "activeUser99", status: "active" },
  { username: "spamBot01", status: "banned" },
  { username: "spamBot02", status: "banned" }
])
```

3. Instructor Scenario: Removing a Single Account

Scenario: A user requests to permanently delete their account ("activeUser99").

The Query:

```
JavaScript
db.accounts.deleteOne({ username: "activeUser99" })
```

4. Student Exercise

-  **Scenario:** Your system has been hit by spam accounts! You need to clean up the database. Write a single query that will permanently delete ALL accounts where the status is "banned".